Elements of Syntax for Parsing

CS 115B – Spring 2026

James Pustejovsky

Brandeis University

January 16, 2026

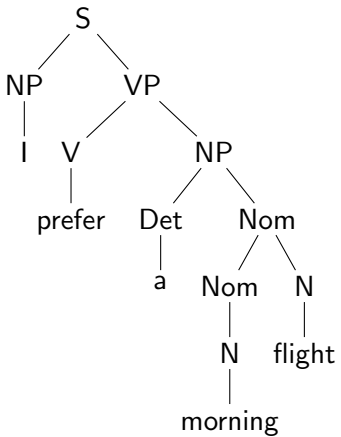
Outline

1. Phrase Structure and Constituency
2. Verb Phrases and Subcategorization
3. Syntactic Ambiguity
4. Context-Free Grammars
5. Parsing as Search
6. CKY Algorithm
7. Earley Algorithm
8. Treebanks
9. Dependency Grammar
10. Probabilistic Parsing (PCFGs)
11. Improving PCFGs
12. Evaluation
13. Summary

Phrase Structure

Phrase structure trees organize sentences into **constituents** (brackets):

- Each constituent gets a label (NP, VP, PP, . . .)
- Constituents are nested in a tree form
- Linguists debate the details
- Lots of ambiguity lurks here!



Constituency Tests

How do we know what nodes go in the tree?

Classic constituency tests:

- **Substitution by proform**: *The big dog* $\rightarrow$ *it*
- **Question answers**: Q: *What did you see?* A: *The big dog*
- **Semantic grounds**: coherence, reference, idioms
- **Dislocation**: *The big dog, I saw it yesterday*
- **Coordination**: *[The cat] and [the dog] ran*

But constituency isn't always clear—tests can conflict!

Conflicting Tests

Phonological Reduction suggests constituency:

- *I will go* → *I'll go*
- *I want to go* → *I wanna go*
- French: *à le centre* → *au centre*

Coordination can create unexpected structures:

- *He went to and came from the store.*

These phenomena challenge simple tree representations.

Verb Phrases

English VPs consist of a **head verb** along with 0 or more following constituents called **arguments**.

Subcategorization: Not all verbs participate in all VP rules.

Verb	Subcategorization Frame
sneeze	John sneezed (intransitive)
find	Please find [a flight to NY]$_{\text{NP}}$
give	Give [me]$_{\text{NP}}$ [a cheaper fare]$_{\text{NP}}$
help	Can you help [me]$_{\text{NP}}$ [with a flight]$_{\text{PP}}$
prefer	I prefer [to leave earlier]$_{\text{TO-VP}}$
told	I was told [United has a flight]$_{\text{S}}$

Programming Analogy

It helps to view verbs as **functions or methods**:

- They specify the **number**, **position**, and **type** of arguments
- Just like formal parameters to a method!

Ungrammatical examples (subcategorization violations):

- **John sneezed the book*
- **I prefer United has a flight*
- **Give with a flight*

Without subcategorization constraints, grammar rules **overgenerate**.

The Ambiguity Problem

Classical NLP Parsing faces severe challenges:

- Minimal grammar on "Fed" sentence: **36 parses**
- Simple, 10-rule grammar: **592 parses**
- Real-size grammar: **many millions of parses**
- Hand-built grammars: ∼30% of sentences have *no* parse

Two fundamental problems:

1. Coverage: Hard to write enough rules for every variation
2. Disambiguation: Many, many parses for each valid sentence

PP Attachment Ambiguity

Consider: *I cleaned the dishes . . .*

- *I cleaned the dishes* **from dinner**. [NP attachment]
- *I cleaned the dishes* **with detergent**. [VP attachment]
- *I cleaned the dishes* **in my pajamas**. [??]
- *I cleaned the dishes* **in the sink**. [??]

Classic example: *They cooked the beans in the pot on the stove with handles.*

Each PP can attach to multiple sites—combinatorial explosion!

More Syntactic Ambiguities

Particle vs. Preposition:

- *The puppy tore up the staircase.*

Complement Structure:

- *The tourists objected to the guide that they couldn't hear.*
- *She knows you like the back of her hand.*

Gerund vs. Participial Adjective:

- *Visiting relatives can be boring.*
- *Changing schedules frequently confused passengers.*

Multiple Gap Constructions:

- *The chicken is ready to eat.*

Phrase Structure Grammars = Context-Free Grammars

Definition: $G = (T, N, S, R)$

- T = set of **terminals** (words)
- N = set of **nonterminals** (phrase categories)
  - Often distinguish $P \subset N$ of *preterminals* (POS tags)
- S = start symbol (usually S for "sentence")
- R = rules/productions of form $X \rightarrow \zeta$
  - $X \in N$, ζ is sequence of terminals and nonterminals

A grammar G **generates** a language L.

A Sample CFG

Phrase Rules:

$$
\begin{aligned}
S &\rightarrow \text{NP VP} \\
VP &\rightarrow \text{V NP} \\
VP &\rightarrow \text{V NP PP} \\
NP &\rightarrow \text{NP PP} \\
NP &\rightarrow \text{N} \\
NP &\rightarrow \text{N N} \\
PP &\rightarrow \text{P NP}
\end{aligned}
$$

Lexical Rules:

$$
\begin{aligned}
N &\rightarrow \textit{cats} \mid \textit{claws} \mid \textit{people} \\
N &\rightarrow \textit{scratch} \\
V &\rightarrow \textit{scratch} \\
P &\rightarrow \textit{with}
\end{aligned}
$$

By convention, S is the start symbol.
In Penn Treebank: extra node at top (ROOT, TOP)

Chomsky Normal Form (CNF)

CNF: All rules have form $X \to YZ$ or $X \to w$

Transformation to CNF doesn't change weak generative capacity:

- Unaries/empties removed recursively
- N-ary rules introduce new nonterminals:

$$\text{VP} \to \text{V NP PP} \quad \Longrightarrow \quad \begin{cases} \text{VP} \to \text{V @VP-V} \\ \text{@VP-V} \to \text{NP PP} \end{cases}$$

Why CNF?

- Makes parsing algorithms simpler and more efficient
- Can reconstruct original trees with bookkeeping

The Parsing Problem

Computational task: Given grammar rules G and sentence w, find a valid parse of w.

Naïve approach: Try all possible trees until finding one that:

- Conforms to grammar rules
- Has "S" at the root
- Has the right words at the leaves

Problem: Takes **exponential time** in number of words!

Parsing can be viewed as:

- Running a grammar backwards
- A search problem
- A hidden data problem

Top-Down vs. Bottom-Up Parsing

Top-Down Parsing

- Start from S, work down to words
- Goal-directed
- Only searches for trees that can be answers
- But: suggests trees not consistent with words
- Problem: left recursion causes infinite loops

Bottom-Up Parsing

- Start from words, work up to S
- Data-directed
- Only forms trees consistent with words
- But: suggests trees that make no sense globally
- Problem: empty categories

Both approaches: independent of depth-first vs. breadth-first search

Human Parsing

Humans exhibit **ambiguity maintenance**:

- *Have the police . . .*
  - *. . . eaten their supper?*
  - *. . . come in and look around.*
  - *. . . taken out and shot.*

But humans also commit early and get **garden-pathed**:

- *The man who hunts ducks out on weekends.*
- *The cotton shirts are made from grows in Mississippi.*
- *The horse raced past the barn fell.*

The Shared Subproblems Issue

Problem: Without caching, backtracking leads to duplicated work.

Example: Parsing "*A flight from Indianapolis to Houston on TWA*"

- Multiple rules for `Nominal`:
  - `Nominal` → `Noun`
  - `Nominal` → `Nominal PP`
- Each backtrack re-parses the same substrings!

Solution: Dynamic Programming

- Fill tables with partial results
- Avoid repeated work
- Solve exponential problems in polynomial time
- Efficiently store ambiguous structures with shared sub-parts

CKY Parsing: Core Idea

Cocke–Kasami–Younger (CKY) algorithm for CNF grammars.

Key insight: For rule $A \to BC$:

- If A spans positions i to j
- Then $\exists k$ such that $i < k < j$:
  - B spans i to k
  - C spans k to j

Build a table: Cell $[i, j]$ contains all nonterminals spanning position i to j.

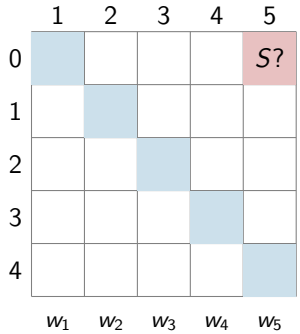
- Complete parse: S in cell $[0, n]$

CKY Algorithm

```
1: for j = 1 to n do
2:     table[j − 1, j] ← {A | A → w_j ∈ R}                    // Lexical lookup
3:     for i = j − 2 down to 0 do
4:         for k = i + 1 to j − 1 do
5:             for each rule A → BC in R do
6:                 if B ∈ table[i, k] and C ∈ table[k, j] then
7:                     add A to table[i, j]
8:                 end if
9:             end for
10:        end for
11:    end for
12: end for
13: return S ∈ table[0, n]
```

Complexity: $O(n^3 \cdot |G|)$ where $|G|$ is grammar size

CKY Table Structure



- Diagonal: lexical entries (POS tags)
- Fill column by column, bottom to top
- Cell $[0, n]$: complete parse if contains S

CKY Notes

Advantages:

- Polynomial time: $O(n^3)$
- Processes input left-to-right ("online")
- When filling a cell, needed parts are already computed

Limitations:

- Bottom-up: imagines "silly" constituents
  - Segments that are constituents but can't occur in context
- Requires CNF grammar
- Returns *recognition*, not full parse trees (without augmentation)

Earley Parsing

Earley's algorithm: Top-down with bottom-up filtering

- Allows **arbitrary CFGs** (no CNF required)
- Single left-to-right sweep over input
- Table of length $N + 1$ (for N words)

Table entries (states) represent:

- Completed constituents and their locations
- In-progress constituents
- Predicted constituents

State notation (dotted rules): $A \rightarrow \alpha \bullet \beta, [i, j]$

- α has been recognized, β still expected
- Spans positions i to j

Earley: Three Operations

1. **Predictor** (top-down prediction)
   - State has nonterminal after dot: $A \rightarrow \alpha \bullet B\beta$
   - Add new states: $B \rightarrow \bullet\gamma$ for each B-rule

2. **Scanner** (match input word)
   - State has POS tag after dot: $A \rightarrow \alpha \bullet \mathit{word}\beta$
   - If next word matches, advance dot

3. **Completer** (bottom-up completion)
   - State is complete: $B \rightarrow \gamma\bullet$
   - Find states looking for B, advance their dots

Success: $S \rightarrow \alpha\bullet, [0, N+1]$ in final column

Earley Example: "Book that flight"

Chart[0]

- $S \rightarrow \bullet$VP
- VP $\rightarrow \bullet$V NP
- V $\rightarrow \bullet$*book*

Chart[1] (after "book")

- V $\rightarrow$ *book*$\bullet$
- VP $\rightarrow$ V $\bullet$ NP
- NP $\rightarrow \bullet$Det Nom
- Det $\rightarrow \bullet$*that*

Chart[3] (after "flight")

- Nom $\rightarrow$ N$\bullet$
- NP $\rightarrow$ Det Nom$\bullet$
- VP $\rightarrow$ V NP$\bullet$
- $S \rightarrow$ VP$\bullet$ ✓

Success: Complete S spanning $[0, 3]$!

Earley: Properties

Complexity:

- $O(n^3)$ for ambiguous grammars
- $O(n^2)$ for unambiguous grammars
- $O(n)$ for bounded-state grammars (e.g., regular)

Advantages over CKY:

- No CNF conversion needed
- Handles left recursion gracefully
- Top-down filtering avoids some spurious constituents

Converting to parser: Add back-pointers in Completer to recover parse trees.

Treebanks

Treebanks are corpora where each sentence is paired with its parse tree.

Creation process:

1. Parse collection with automatic parser
2. Human annotators hand-correct each parse
3. Requires detailed annotation guidelines

Penn Treebank (most widely used):

- 1M words from Wall Street Journal (1987–1989)
- Standard train/test splits for evaluation
- $\sim$40,000 training sentences, $\sim$2,400 test sentences

Treebank Grammars

Treebanks **implicitly define a grammar**:

- Extract local rules from all subtrees
- WSJ section yields ∼12,000 rules

Characteristics:

- Very **flat** trees (avoid deep recursion for annotator ease)
- Penn Treebank: ∼4,500 different VP rules!

Uses:

- Training statistical parsers
- Corpus linguistics: investigating constructions
- Evaluating parser performance

Head Finding

Head words are critical for statistical parsing.

- Each phrase has a lexical **head**
- Simple rules identify heads by traversing parse trees

Head-finding rules:

- Head of VP = main verb
- Head of NP = main noun
- Head of PP = preposition
- Head of S = head of VP

Lexically decorated tree: Each node annotated with its head word.

Dependency Grammar

Alternative to phrase structure: Relations between words, not constituents.

Dependency parse:

- Nodes = words in the utterance
- Links = dependency relations (may be labeled/typed)



Dependency Parsing: Advantages

Advantages over phrase-structure parsing:

- Handles **free word order** languages well
- **Faster** parsing than CFG-based parsers
- Often captures syntactic relations needed by downstream applications
  - CFG parsers often extract this same information anyway!

Can convert phrase structure → dependencies:

- Make head of each non-head child depend on head of head child
- Uses head-finding rules

Probabilistic Context-Free Grammars

PCFG: $G = (T, N, S, R, P)$

- Same as CFG, plus $P(R)$ gives probability of each rule
- Probabilities for rules with same LHS sum to 1:

$$\forall X \in N : \sum_{X \to \zeta \in R} P(X \to \zeta) = 1$$

Probability of a tree:

$$P(t) = \prod_{(X \to \zeta) \in t} P(X \to \zeta)$$

Probability of a string:

$$P(w_1^n) = \sum_{t \text{ parses } w_1^n} P(t)$$

PCFG Example

$S \rightarrow$ NP VP	1.0
$VP \rightarrow$ V NP	0.7
$VP \rightarrow$ VP PP	0.3
$PP \rightarrow$ P NP	1.0
$NP \rightarrow$ NP PP	0.4
$NP \rightarrow$ *astronomers*	0.1
$NP \rightarrow$ *stars*	0.18
$NP \rightarrow$ *ears*	0.18

Sentence: *astronomers saw stars with ears*

Two parses:

- $P(t_1)$ [PP attaches to NP] $= 0.0009072$
- $P(t_2)$ [PP attaches to VP] $= 0.0006804$

$P(\text{sentence}) = P(t_1) + P(t_2) = 0.0015876$

Parser prefers t_1 (NP attachment)

Probabilistic CKY

Modify CKY to track probabilities:

- Cell$[i, j]$ stores: nonterminal + **probability of best derivation**
- When combining B from $[i, k]$ and C from $[k, j]$:

$$P(A, [i, j]) = P(A \rightarrow BC) \cdot P(B, [i, k]) \cdot P(C, [k, j])$$

- Take **max** over all ways to derive each constituent

Viterbi-style algorithm: Find most probable parse tree.

Inside algorithm: Sum probabilities (for computing P(sentence)).

Training PCFGs

Supervised training (from treebank):

- Rules: extracted directly from trees
- Probabilities: estimated from counts

$$P(A \to \zeta) = \frac{\text{count}(A \to \zeta)}{\text{count}(A)}$$

Unsupervised training (Inside-Outside algorithm):

- EM algorithm for PCFGs (like Forward-Backward for HMMs)
- Given number of nonterminals, learn rule probabilities
- Results historically not impressive
- Recent work adds constraints to improve learning

Vanilla PCFG Limitations

Problem: Probabilities don't depend on specific words.

- Can only learn general structural preferences
- Example: "prefer PP attachment to NPs"

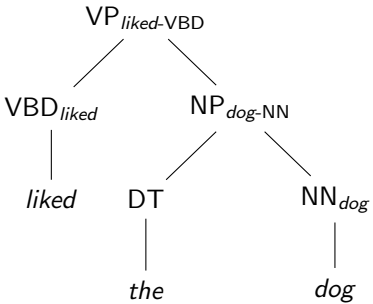
But disambiguation often requires lexical information:

- *I ate spaghetti* **with** *a fork.* [VP attachment]
- *I ate spaghetti* **with** *meatballs.* [NP attachment]

Solution: **Lexicalization**—specialize productions to specific words by including head-words in nonterminal symbols.

Lexicalized Productions

Include head word and POS in each nonterminal:



Lexicalized rule:

$$\text{VP}_{\textit{liked}\text{-VBD}} \rightarrow \text{VBD}_{\textit{liked}}\ \text{NP}_{\textit{dog}\text{-NN}}$$

Collins Parser

Problem: Lexicalized productions create data sparsity.

Collins (1999) solution: Generative model of productions

- Model productions based on context left/right of head
- LHS $\rightarrow L_n \ldots L_1\ H\ R_1 \ldots R_m$
- Generate head H, then generate left/right context until STOP

Smoothing: Interpolate with simpler models:

$$\tilde{P}(R|\text{VP}_{put}) = \lambda_1 P(R|\text{VP}_{put}) + (1-\lambda_1)[\lambda_2 P(R|\text{VP}_{\text{VBD}}) + (1-\lambda_2)P(R|\text{VP})]$$

Parent Annotation

Problem: CFG independence assumption violated—production choice depends on context.

- Subject NPs more likely to be pronouns than object NPs

Solution: **Split nonterminals** based on parent:

- Subject NP $\rightarrow$ NP^S
- Object NP $\rightarrow$ NP^VP

Split and Merge:

- Splitting increases grammar size $\rightarrow$ data sparsity
- Only split when it improves accuracy
- Merge unhelpful distinctions
- Search for split/merge combination maximizing treebank likelihood

PARSEVAL Evaluation Metrics

PARSEVAL metrics: Compare computed parse P to gold standard T.

Labeled metrics (must match constituent label):

$$\text{Recall} = \frac{\#\text{ correct constituents in } P}{\#\text{ constituents in } T}$$

$$\text{Precision} = \frac{\#\text{ correct constituents in } P}{\#\text{ constituents in } P}$$

F1 score (harmonic mean):

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

State-of-the-art: >90% labeled F1 on Penn WSJ

Discriminative Parse Reranking

Observation: Correct parse often in top-N even when not #1.

Reranking approach:

1. PCFG parser produces N-best list
2. Extract **global features** from each parse
3. Train discriminative classifier (e.g., logistic regression)
4. Rerank using classifier scores

Sample features:

- PCFG probability
- Number of parallel conjuncts
- Degree of right-branching
- Frequency of tree fragments

Improvement: 1–2% over base PCFG model

Summary

Key concepts:

- **CFGs** model syntactic structure; **PCFGs** add probabilities
- **Parsing** = search for valid tree structures
- **Dynamic programming** (CKY, Earley) enables polynomial-time parsing
- **Treebanks** provide supervised training data

Improving PCFGs:

- Lexicalization captures word-specific preferences
- Parent annotation provides context
- Discriminative reranking uses global features

Alternatives:

- Dependency parsing: relations between words
- Unification grammars (HPSG, LFG): richer feature structures